Cuadernillo Dolphin Smalltalk

Estructuras de decisión.

```
(condición)ifTrue:[bloque de código].
(condición)ifFalse:[bloque de código].
(condición)ifTrue:[bloque de código]ifFalse:[bloque de código].
```

Estructuras de repetición.

```
[condición]whileTrue:[bloque de código].
[condición]whileFalse:[bloque de código].
```

Conversión de datos.

Dependiendo del contexto, podemos llegar a necesitar convertir de un tipo de dato a otro. Cuando necesitamos convertir una cadena de caracteres a un número para imprimirlo en el Transcript, lo podemos hacer con el método **displayString**.

```
Trasncript show: remedio verPrecio displayString.
```

Por otro lado también está el caso en el que recibimos un string y queremos convertirlo a número. Para esto utilizo el método **asNumber**.

```
precio := (Prompter prompt: 'Ingrese el precio del remedio')asNumber.
```

El método asNumber distingue a qué tipo de número tiene que convertir la cadena de caracteres.

Implementando una clase en el entorno.

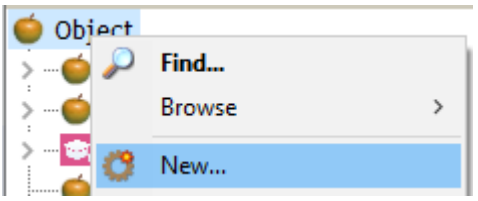
Paso 1: Abrir el class browser



Debemos buscar este logo que es el segundo ícono del entorno (el ícono puede variar segun la versión pero se seguirá llamando igual.

Paso 2: Crear la clase.

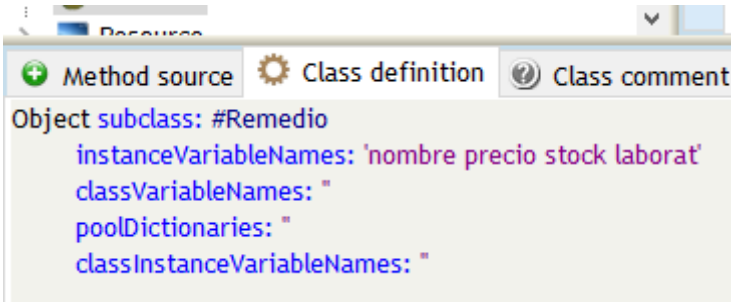
Para crear la clase debemos hacer click derecho en la clase que queramos usar como clase padre y seleccionar la opción new. Esto va a generar una nueva clase vacía que debemos cargar con sus datos, por ahora nos centraremos en definir las variables y métodos ya sean de instancia o clase.



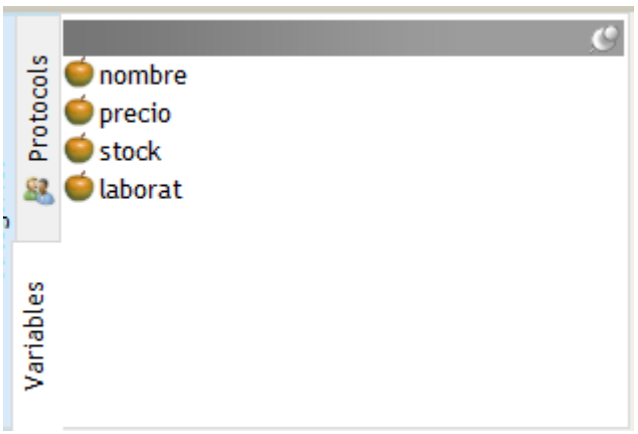
Paso 3: Agregarle atributos.

Se pueden agregar atributos a una clase utilizando la pestaña “Class Definition” del menú inferior. Dentro de esta pestaña tenemos todo lo que es la definición de la clase.
Entre la definición podemos encontrar las líneas “instanceVariableNames: ‘’ “ y “classVariableNames: ‘’ “ (Mayormente nos vamos a concentrar en las variables de instancia pero puede llegar a darse el caso en el que necesitemos definir una de clase).
Para darle atributos a la clase debemos escribir entre las comillas simples los nombres de los mismos separados por espacios, como se muestra a continuación:

instanceVariableNames: ‘atributo1 atributo2 atributo3 … atributoN’

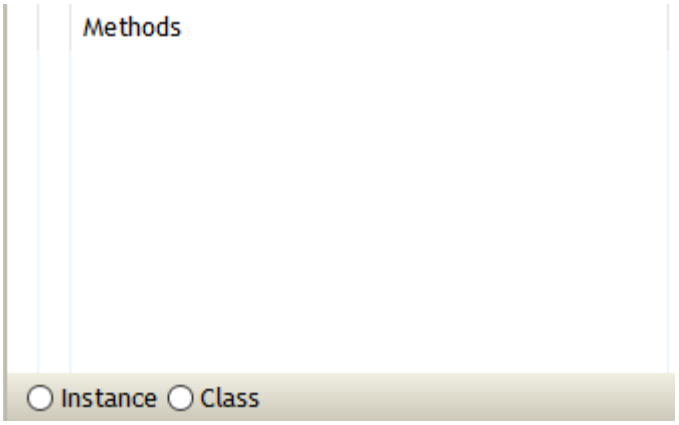


Usando “Ctrl + s” para guardar los cambios, vamos a ver reflejados los atributos que creamos en la pestaña del medio



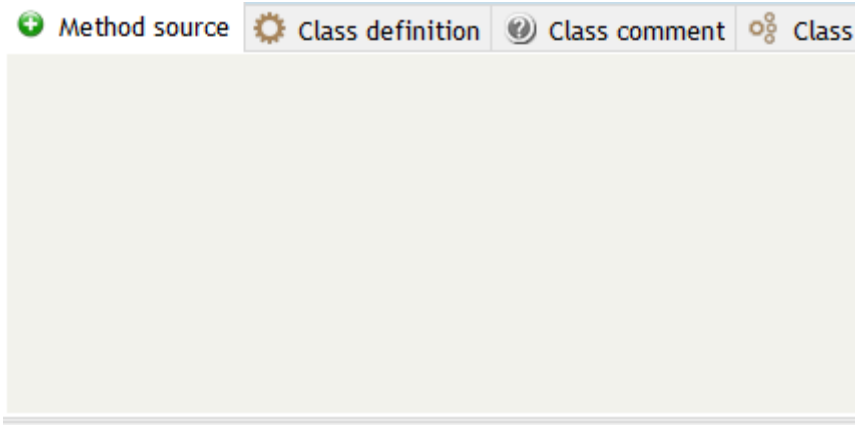
Paso 4: Agregando métodos.

Los métodos se listan en la pestaña de arriba a la derecha (La tercer pestaña de la parte superior). Abajo de esta pestaña hay un selector de radio que permite cambiar entre instancia y clase.

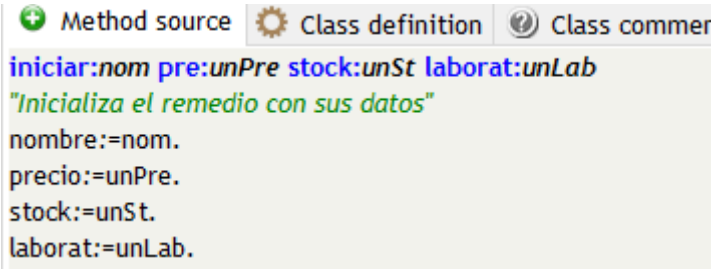


Este selector va a permitirnos elegir que tipo de método crear.

Después para crear un método vamos a la pestaña de abajo donde dice “Method Source”. Este espacio es para escribir la implementación de los métodos.



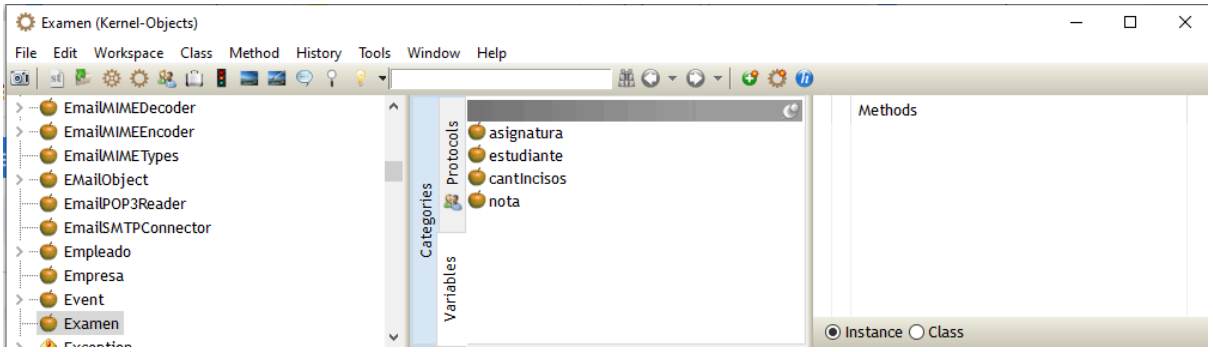
La primera línea que se escribe viene a ser el método, con el selector y los distintos argumentos. Esta línea **NO** lleva punto al final. El resto de las líneas serán la implementación del método. Cada sentencia debe ir separada por punto.
A continuación se muestra un ejemplo de como escribir el método.



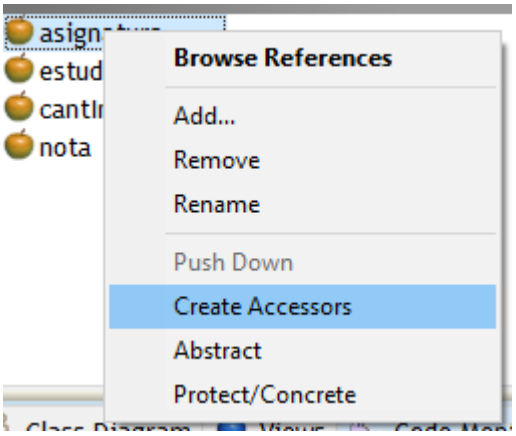
Para guardar, simplemente se presiona “Ctrl + s” y se verá reflejado en la ventana que se menciona previamente donde se listan los métodos.

Creando los ver y modificar (getters y setters).

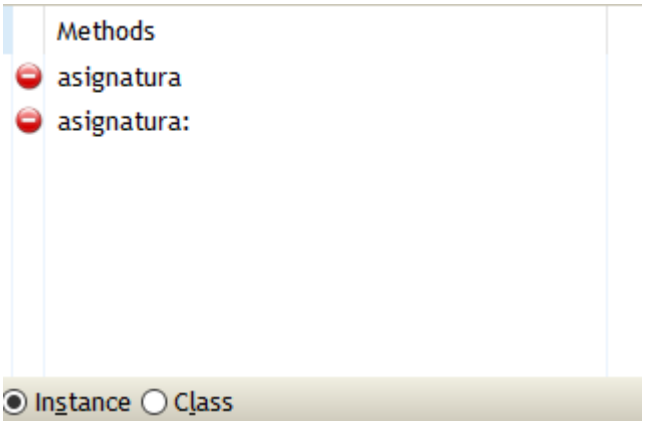
Existe una forma automática de resolverlo. Teniendo una clase con sus atributos creados. podemos hacer click derecho en el atributo que queramos (o seleccionar todos) y elegir la opción “Create Accesors”, como se muestra a continuación.



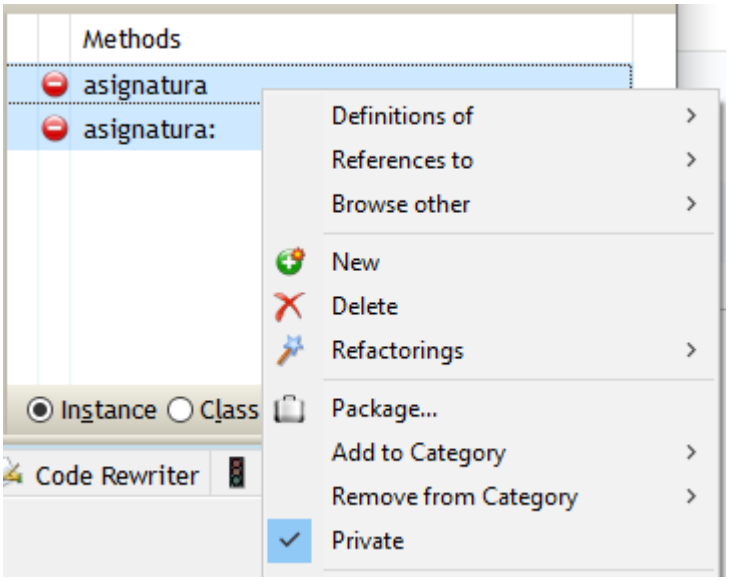
Seleccionamos asignatura y hacemos click derecho:



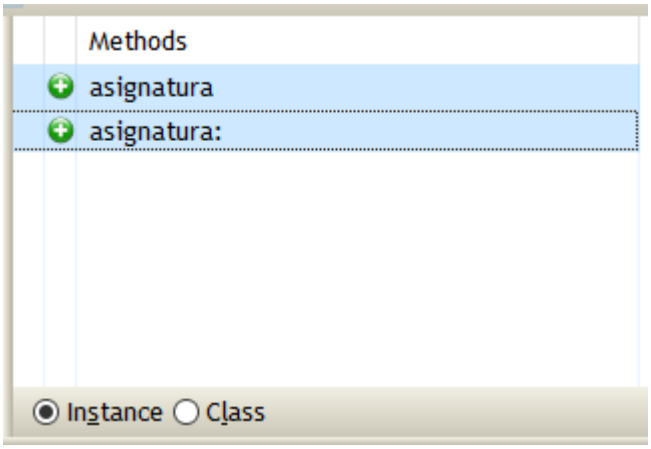
Los vamos a ver reflejados a la derecha, donde dice “Methods”



Pero quedan definidos como métodos privados. Podemos resolverlo seleccionandolos, haciendo click derecho y seleccionando la opción “Private” la cual debería tener un tick a la izquierda actualmente.



Al presionarlo, el tick desaparece y los métodos quedan como públicos (denotado por un circulo verde con un más a la izquierda del nombre).



Mensajes en cascada.

A la hora de escribir código, se puede volver repetitivo el uso de una clase. Por ejemplo, puede que al escribir en el Transcript ocupemos varias líneas de código para imprimir poco texto.

```
Transcript clear.
Transcript show: ‘2024’.
Transcript cr.
Transcript show: ‘Actividad n°5’.
Transcript cr.
Transcript show: ‘Presentación de ejercicios de Herencia’.
Transcript cr.
```

En este caso llamamos 7 veces a la clase Transcript. Los mensajes en cascada nos permiten colocar el objeto receptor una única vez y enviarle distintos mensajes en una sola línea.

```
Transcript clear; show:’2024’; cr; show: ‘Actividad n°5’; cr;
show: ‘Presentación de ejercicios de Herencia’; cr.
```

De esta forma llamamos una única vez a la clase Transcript y en una línea escribimos lo mismo que en el ejemplo anterior. No hay que abusar mucho de los mensajes en cascada, ya que si bien permiten agilizar la escritura, pueden afectar la legibilidad del código.

Métodos útiles

Ver la clase a la que pertenece un objeto:

`obj class.`

(Siendo “obj” una variable que guarda un objeto ya instanciado y “class” el método que devuelve el nombre de la clase).

Por ejemplo:

```
1 class. “Esto retorna SmallInteger”
```

Ver si un objeto pertenece a determinada clase

Para comprobar si un método pertenece a una clase particular se puede utilizar ***isKindOf***.
Ejemplo

```
1 isKindOf: SmallInteger.
```

Tipos de datos útiles

Date

El tipo date es la forma de manejar las fechas en Dolphin Smalltalk. Para crear una nueva fecha, podemos utilizar varios métodos, el más común sería:

```
Date newDay: day monthIndex: month year: year.
```

Que crea una fecha nueva recibiendo 3 enteros día, mes y año.

Algunas operaciones para elementos de tipo Date son:

Método	Resultado	Descripción
<	bool	"Menor que"
>	bool	"Mayor que"
monthsSince:fecha	int	"Meses transcurridos desde la fecha dada"
yearsSince:fecha	int	"Años transcurridos desde la fecha dada"
weekday	string	"Nombre del día"
monthName	string	"Nombre del mes"

```
d1:= Date newDay: 30 monthIndex: 12 year: 2000.
d2:= Date newDay: 30 monthIndex: 12 year: 2023.

d1 < d2. "true"
d1 > d2. "false"
d2 monthsSince: d1."276"
d2 yearsSince: d1."23"
d1 weekday."'sábado'"
d2 monthName."'diciembre'"
```

Estructuras de datos

Ejercicios ejemplo.

Aclaraciones.